

```
=====
MONGO ARCHIVER - Spring Boot (v4)
Configurable cross-collection MongoDB archiver
Single audit log entry per entity | Temporary index management
=====
```

```
=====
PROJECT STRUCTURE
=====
```

```
mongo-archiver-java-v4/
├─ pom.xml
├─ src/main/
│   ├─ resources/
│   │   └─ application.yml          ← Only file you edit for new entity
│   └─ types
│       └─ java/com/archiver/
│           ├─ MongoArchiverApplication.java    ← Entry point + --run-now flag
│           ├─ config/
│           │   └─ ArchiverProperties.java      ← @ConfigurationProperties binding
│           ├─ core/
│           │   └─ Archiver.java                ← Core archive engine
│           ├─ job/
│           │   └─ ArchiveCronJob.java          ← @Scheduled cron trigger
│           ├─ model/
│           │   └─ ArchiveLogEntry.java         ← One doc per entity in archiveLog
│           └─ util/
│               ├─ ArchiveLogger.java           ← Writes one entry per entity
│               ├─ IdPathResolver.java          ← Dot-notation query builder
│               └─ IndexManager.java            ← Temporary index lifecycle
```

```
=====
HOW IT WORKS
=====
```

1. TEMPORARY INDEXES

Before any batching, IndexManager ensures an index on every queried field:

- anchor.createdTsField (cutoff query)
- anchor.idField (per-entity lookup)
- each related.idPath (per-entity lookup)

Pre-existing indexes → reused, never dropped.

Newly created indexes → prefixed `_arch_tmp_`, dropped in finally block.

Guaranteed cleanup even if the run crashes mid-way.

2. ANCHOR ID FETCH

Query anchor collection for records where `createdTsField < now - archiveAfterDays`.

Only the anchor uses time as the gate. Related collections are looked up by ID only.

3. BATCH PROCESSING

Entity IDs split into chunks of `batchSize`. One entity fully processed before next.

4. PER ENTITY – related first, anchor last

a. Archive each related collection in order.

b. If ALL related succeeded → archive + delete anchor.

c. If ANY related failed → mark anchor as SKIPPED, leave it in source DB.

Next `createdTs` sweep naturally retries this entity.

5. SINGLE AUDIT LOG ENTRY PER ENTITY

After all collections are processed, ONE document is upserted into `archiveLog` keyed on `(jobName, entityId)`. It contains the rolled-up status and a per-collection result map.

```
=====
AUDIT LOG – DOCUMENT SHAPE
=====
```

One document per entity per run. Upserted – re-runs overwrite previous entry.

```
{
  "jobName"    : "orders",
  "entityId"   : "ORD-7823",
  "status"     : "SUCCESS | PARTIAL_FAILURE | FAILED",
  "collections": {
    "orderItems"       : "SUCCESS | FAILED | NO_DOCUMENTS",
    "orderPayments"    : "SUCCESS | FAILED | NO_DOCUMENTS",
    "orderEvents"      : "SUCCESS | FAILED | NO_DOCUMENTS",
    "orderNotifications": "SUCCESS | FAILED | NO_DOCUMENTS",
    "orders"           : "SUCCESS | FAILED | NO_DOCUMENTS | SKIPPED"
  },
  "runAt"       : ISODate("..."),
  "retryCount"  : 1,
  "createdAt"   : ISODate("...")
}
```

Overall status derivation:

All SUCCESS / NO_DOCUMENTS	→ SUCCESS
Any SKIPPED (anchor not attempted)	→ PARTIAL_FAILURE
Some FAILED, some succeeded	→ PARTIAL_FAILURE
All FAILED	→ FAILED

Example documents:

-- Full success --

```
{ "entityId":"ORD-7823", "status":"SUCCESS",
  "collections": { "orderItems":"SUCCESS", "orderPayments":"SUCCESS",
                   "orderEvents":"NO_DOCUMENTS", "orderNotifications":"SUCCESS",
                   "orders":"SUCCESS" } }
```

-- Partial failure (orderNotifications failed, anchor skipped) --

```
{ "entityId":"ORD-7824", "status":"PARTIAL_FAILURE",
  "collections": { "orderItems":"SUCCESS", "orderPayments":"SUCCESS",
                   "orderEvents":"NO_DOCUMENTS", "orderNotifications":"FAILED",
                   "orders":"SKIPPED" } }
```

-- Re-run after fix (retryCount increments) --

```
{ "entityId":"ORD-7824", "status":"SUCCESS", "retryCount":2,
  "collections": { "orderItems":"SUCCESS", "orderPayments":"SUCCESS",
                   "orderEvents":"NO_DOCUMENTS", "orderNotifications":"SUCCESS",
                   "orders":"SUCCESS" } }
```

```
=====
MONITORING QUERIES
=====
```

// All partial/full failures

```
db.archiveLog.find({ status: { $in: ["FAILED","PARTIAL_FAILURE"] } }).sort({
runAt: -1 })
```

// Entities where a specific collection failed

```
db.archiveLog.find({ "collections.orderNotifications": "FAILED" })
```

// Full history for one entity

```
db.archiveLog.find({ entityId: "ORD-7823" }).sort({ runAt: -1 })
```

// Count by status for a job

```
db.archiveLog.aggregate([
  { $match: { jobName: "orders" } },
  { $group: { _id: "$status", count: { $sum: 1 } } }
])
```

// Leftover temporary indexes after a crash

```
db.orders.getIndexes().filter(i => i.name.startsWith("_arch_tmp_"))
```

```
=====
RUNNING
=====
```

```
mvn clean package -DskipTests
```

```
# Scheduled mode
```

```
java -jar target/mongo-archiver-1.0.0.jar
```

```
# One-shot / backfill
```

```
java -jar target/mongo-archiver-1.0.0.jar --run-now
```

```
# Override connection
```

```
MONGO_URI=mongodb://prod:27017 MONGO_DB=mydb java -jar target/mongo-archiver-1.0.0.jar
```

```
=====
FILE: pom.xml
=====
```

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.2.3</version>
    <relativePath/>
  </parent>
  <groupId>com.archiver</groupId>
  <artifactId>mongo-archiver</artifactId>
  <version>1.0.0</version>
  <properties>
    <java.version>17</java.version>
  </properties>
  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter</artifactId>
    </dependency>
```

```

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-mongodb</artifactId>
</dependency>
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <optional>true</optional>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-configuration-processor</artifactId>
  <optional>true</optional>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
</dependencies>
<build>
  <plugins>
    <plugin>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-maven-plugin</artifactId>
      <configuration>
        <excludes>
          <exclude>
            <groupId>org.projectlombok</groupId>
            <artifactId>lombok</artifactId>
          </exclude>
        </excludes>
      </configuration>
    </plugin>
  </plugins>
</build>
</project>

```

```

=====
FILE: src/main/resources/application.yml
=====

```

```

spring:
  data:
    mongodb:
      uri: ${MONGO_URI:mongodb://localhost:27017}
      database: ${MONGO_DB:appdb}

```

```
archiver:
  schedule: "0 0 2 * * ?"
  timezone: UTC
  archive-log-collection: archiveLog
  jobs:
    - name: orders
      batch-size: 100
      anchor:
        collection: orders
        id-field: orderId
        created-ts-field: createdAt
        archive-after-days: 90
        delete-after-archive: true
        archive-collection: orders_archive
      related:
        - collection: orderItems
          id-path: orderId
          delete-after-archive: true
          archive-collection: orderItems_archive
        - collection: orderPayments
          id-path: order.id
          delete-after-archive: true
          archive-collection: orderPayments_archive
        - collection: orderEvents
          id-path: meta.referenceId
          delete-after-archive: false
          archive-collection: orderEvents_archive
        - collection: orderNotifications
          id-path: payload.orderId
          delete-after-archive: true
          archive-collection: orderNotifications_archive
```

```
logging:
  level:
    com.archiver: INFO
  file:
    name: logs/archiver.log
    max-size: 10MB
    max-history: 5
```

```
=====
FILE: src/main/java/com/archiver/MongoArchiverApplication.java
=====
```

```
package com.archiver;
```

```

import com.archiver.job.ArchiveCronJob;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.boot.ApplicationArguments;
import org.springframework.boot.ApplicationRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.scheduling.annotation.EnableScheduling;

@Slf4j
@SpringBootApplication
@EnableScheduling
@RequiredArgsConstructor
public class MongoArchiverApplication implements ApplicationRunner {

    private final ArchiveCronJob archiveCronJob;

    public static void main(String[] args) {
        SpringApplication.run(MongoArchiverApplication.class, args);
    }

    @Override
    public void run(ApplicationArguments args) {
        if (args.containsOption("run-now")) {
            log.info("[MongoArchiverApplication] --run-now flag detected;
executing immediately");
            archiveCronJob.runNow();
            log.info("[MongoArchiverApplication] One-shot run complete – shutting
down");
            System.exit(0);
        }
        log.info("[MongoArchiverApplication] Running in scheduled mode");
    }
}

```

```

=====
FILE: src/main/java/com/archiver/config/ArchiverProperties.java
=====

```

```

package com.archiver.config;

import lombok.Data;
import org.springframework.boot.context.properties.ConfigurationProperties;
import org.springframework.stereotype.Component;

import java.util.List;

```

```

@Data
@Component
@ConfigurationProperties(prefix = "archiver")
public class ArchiverProperties {

    private String schedule = "0 0 2 * * ?";
    private String timezone = "UTC";
    private String archiveLogCollection = "archiveLog";
    private List<JobProperties> jobs;

    @Data
    public static class JobProperties {
        private String name;
        private int batchSize = 100;
        private AnchorProperties anchor;
        private List<RelatedProperties> related;
    }

    @Data
    public static class AnchorProperties {
        private String collection;
        private String idField;
        private String createdTsField;
        private int archiveAfterDays;
        private boolean deleteAfterArchive = true;
        private String archiveCollection;
    }

    @Data
    public static class RelatedProperties {
        private String collection;
        private String idPath;
        private boolean deleteAfterArchive = true;
        private String archiveCollection;
    }
}

```

```

=====
FILE: src/main/java/com/archiver/core/Archiver.java
=====

```

```

package com.archiver.core;

import com.archiver.config.ArchiverProperties;
import com.archiver.config.ArchiverProperties.AnchorProperties;
import com.archiver.config.ArchiverProperties.JobProperties;
import com.archiver.config.ArchiverProperties.RelatedProperties;

```



```

import com.archiver.model.ArchiveLogEntry;
import com.archiver.model.ArchiveLogEntry.CollectionStatus;
import com.archiver.model.ArchiveLogEntry.Status;
import com.archiver.util.ArchiveLogger;
import com.archiver.util.IdPathResolver;
import com.archiver.util.IndexManager;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.bson.Document;
import org.springframework.data.mongodb.core.BulkOperations;
import org.springframework.data.mongodb.core.MongoTemplate;
import org.springframework.data.mongodb.core.query.Query;

import java.util.*;

/**
 * Archiver
 *
 * <p>Core archive engine for one configured job.
 *
 * <h3>Per-entity audit log</h3>
 * <p>One {@code archiveLog} document is written per entity at the end of
processing.
 * It contains a {@code collections} map with each collection's outcome, and a
 * rolled-up {@code status}:
 * <ul>
 * <li>{@code SUCCESS} - all collections succeeded or had no
documents</li>
 * <li>{@code PARTIAL_FAILURE} - at least one failed, at least one
succeeded</li>
 * <li>{@code FAILED} - all related collections failed</li>
 * </ul>
 * The anchor entry in the map is {@code SKIPPED} if any related collection
failed.
 *
 * <h3>Index strategy</h3>
 * <p>Temporary indexes are created before batching and dropped in a {@code
finally}
 * block - guaranteed cleanup even on mid-run failures.
 */
@Slf4j
@RequiredArgsConstructor
public class Archiver {

    private final JobProperties jobConfig;
    private final MongoTemplate mongoTemplate;
    private final ArchiveLogger archiveLogger;

```

// — Entry point —————

```
public void run() {
    String jobName = jobConfig.getName();
    Date cutoff = getCutoffDate(jobConfig.getAnchor().getArchiveAfterDays());

    log.info("[Archiver:{}] Starting — archiving records with {} < {}",
        jobName, jobConfig.getAnchor().getCreatedTsField(), cutoff);

    IndexManager indexManager = new IndexManager(mongoTemplate);

    try {
        ensureIndexes(indexManager);

        List<String> ids = fetchAnchorIds(cutoff);
        if (ids.isEmpty()) {
            log.info("[Archiver:{}] No records to archive", jobName);
            return;
        }

        log.info("[Archiver:{}] {} entities to process", jobName, ids.size());

        int batchSize = jobConfig.getBatchSize();
        int totalBatches = (int) Math.ceil((double) ids.size() / batchSize);

        for (int i = 0; i < ids.size(); i += batchSize) {
            List<String> batch = ids.subList(i, Math.min(i + batchSize,
ids.size()));
            log.info("[Archiver:{}] Batch {}/{ } ({} IDs)",
                jobName, (i / batchSize) + 1, totalBatches, batch.size());
            processBatch(batch);
        }

        log.info("[Archiver:{}] Archive run complete", jobName);

    } finally {
        indexManager.dropCreatedIndexes();
    }
}
```

// — Index management —————

```
private void ensureIndexes(IndexManager indexManager) {
    AnchorProperties anchor = jobConfig.getAnchor();
    indexManager.ensureIndex(anchor.getCollection(),
anchor.getCreatedTsField());
}
```

```

        indexManager.ensureIndex(anchor.getCollection(), anchor.getIdField());
        for (RelatedProperties rel : jobConfig.getRelated()) {
            indexManager.ensureIndex(rel.getCollection(), rel.getIdPath());
        }
        log.info("[Archiver:{}] Index check complete - {} temporary index(es)
created",
            jobConfig.getName(), indexManager.ownedCount());
    }

    // — Batch processing —————

    private void processBatch(List<String> entityIds) {
        for (String entityId : entityIds) {
            processEntity(entityId);
        }
    }

    /**
     * Archive all collections for one entity, then write a single audit log
    entry.
     *
     * <p>The {@code collectionResults} map is built in order:
     * related collections first, anchor last (or SKIPPED if a related one
    failed).
     */
    private void processEntity(String entityId) {
        // LinkedHashMap preserves insertion order in the log document
        Map<String, String> collectionResults = new LinkedHashMap<>();

        // — Archive related collections —————
        boolean allRelatedSucceeded = true;
        for (RelatedProperties rel : jobConfig.getRelated()) {
            CollectionStatus result = archiveCollection(
                entityId,
                rel.getCollection(),
                rel.getIdPath(),
                rel.getArchiveCollection(),
                rel.isDeleteAfterArchive()
            );
            collectionResults.put(rel.getCollection(), result.name());
            if (result == CollectionStatus.FAILED) {
                allRelatedSucceeded = false;
            }
        }

        // — Archive anchor — only if all related succeeded —————
        AnchorProperties anchor = jobConfig.getAnchor();

```

```

        if (allRelatedSucceeded) {
            CollectionStatus anchorResult = archiveCollection(
                entityId,
                anchor.getCollection(),
                anchor.getIdField(),
                anchor.getArchiveCollection(),
                anchor.isDeleteAfterArchive()
            );
            collectionResults.put(anchor.getCollection(), anchorResult.name());
        } else {
            collectionResults.put(anchor.getCollection(),
CollectionStatus.SKIPPED.name());
            log.warn("[Archiver:{}] Anchor skipped for entity '{}' - related
failure. " +
                "Will retry on next createdTs sweep.", jobConfig.getName(),
entityId);
        }

        // — Write one log entry for this entity —————
        archiveLogger.log(ArchiveLogEntry.builder()
            .jobName(jobConfig.getName())
            .entityId(entityId)
            .status(deriveOverallStatus(collectionResults))
            .collections(collectionResults)
            .runAt(new Date())
            .build());
    }

    // — Overall status derivation —————

    /**
     * Roll up per-collection results into a single entity-level status.
     *
     * <ul>
     *   <li>Any SKIPPED present → PARTIAL_FAILURE (anchor was not archived)</li>
     *   <li>All FAILED → FAILED</li>
     *   <li>Any FAILED → PARTIAL_FAILURE</li>
     *   <li>Otherwise → SUCCESS</li>
     * </ul>
     */
    private Status deriveOverallStatus(Map<String, String> results) {
        long failed = results.values().stream().filter(s ->
s.equals(CollectionStatus.FAILED.name())).count();
        long skipped = results.values().stream().filter(s ->
s.equals(CollectionStatus.SKIPPED.name())).count();

        if (skipped > 0)
            return Status.PARTIAL_FAILURE;
    }

```

```

        if (failed == results.size())        return Status.FAILED;
        if (failed > 0)                      return Status.PARTIAL_FAILURE;
        return Status.SUCCESS;
    }

    // — Core archive operation —————

    /**
     * Find all documents for {@code entityId} in {@code srcCollection},
     * write to {@code archiveCollection}, optionally delete from source.
     *
     * @return the {@link CollectionStatus} for this operation
     */
    private CollectionStatus archiveCollection(
        String entityId,
        String srcCollection,
        String idPath,
        String archiveCollection,
        boolean deleteAfterArchive
    ) {
        String jobName = jobConfig.getName();
        try {
            Query query = IdPathResolver.buildSingleQuery(idPath, entityId);
            List<Document> docs = mongoTemplate.find(query, Document.class,
srcCollection);

            if (docs.isEmpty()) {
                log.debug("[Archiver:{}] No documents in '{}' for entity '{}'",
                    jobName, srcCollection, entityId);
                return CollectionStatus.NO_DOCUMENTS;
            }

            Date archivedAt = new Date();
            docs.forEach(doc -> {
                doc.append("_archivedAt", archivedAt);
                doc.append("_archivedFrom", srcCollection);
            });

            bulkInsert(docs, archiveCollection, jobName, entityId, srcCollection);

            if (deleteAfterArchive) {
                mongoTemplate.remove(query, srcCollection);
            }

            log.info("[Archiver:{}] Archived {} doc(s): '{}' → '{}' (entity:
'{}')",
                jobName, docs.size(), srcCollection, archiveCollection, entityId);

```

```

        return CollectionStatus.SUCCESS;

    } catch (Exception e) {
        log.error("[Archiver:{}] FAILED '{}' for entity '{}': {}",
            jobName, srcCollection, entityId, e.getMessage(), e);
        return CollectionStatus.FAILED;
    }
}

// — Bulk insert with duplicate tolerance —————

private void bulkInsert(
    List<Document> docs, String collection,
    String jobName, String entityId, String srcCollection
) {
    try {
        BulkOperations bulk =
mongoTemplate.bulkOps(BulkOperations.BulkMode.UNORDERED, collection);
        docs.forEach(bulk::insert);
        bulk.execute();
    } catch (org.springframework.data.mongodb.BulkOperationException e) {
        int inserted = e.getResult() != null ?
e.getResult().getInsertedCount() : 0;
        int dupes = docs.size() - inserted;
        if (dupes > 0) {
            log.warn("[Archiver:{}] '{}' entity='{}' - {} already archived, {}
newly inserted",
                jobName, srcCollection, entityId, dupes, inserted);
        }
    }
}

// — Anchor ID fetching —————

private List<String> fetchAnchorIds(Date cutoff) {
    AnchorProperties anchor = jobConfig.getAnchor();
    try {
        Query query =
IdPathResolver.buildCutoffQuery(anchor.getCreatedTsField(), cutoff);
        query.fields().include(anchor.getIdField()).exclude("_id");

        Set<String> ids = new LinkedHashSet<>();
        mongoTemplate.find(query, Document.class, anchor.getCollection())
            .forEach(doc -> {
                String id = IdPathResolver.extractIdAsString(doc,
anchor.getIdField());

```

```

        if (id != null) ids.add(id);
    });

    log.info("[Archiver:{}] Found {} anchor IDs in '{}'",
        jobConfig.getName(), ids.size(), anchor.getCollection());
    return new ArrayList<>(ids);

} catch (Exception e) {
    log.error("[Archiver:{}] Failed to fetch anchor IDs from '{}': {}",
        jobConfig.getName(), anchor.getCollection(), e.getMessage(), e);
    return List.of();
}
}

// ——— Helpers —————
private Date getCutoffDate(int days) {
    Calendar cal = Calendar.getInstance();
    cal.add(Calendar.DATE, -days);
    return cal.getTime();
}
}

```

```

=====
FILE: src/main/java/com/archiver/job/ArchiveCronJob.java
=====

```

```

package com.archiver.job;

import com.archiver.config.ArchiverProperties;
import com.archiver.core.Archiver;
import com.archiver.util.ArchiveLogger;
import jakarta.annotation.PostConstruct;
import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.data.mongodb.core.MongoTemplate;
import org.springframework.scheduling.annotation.Scheduled;
import org.springframework.stereotype.Component;

import java.time.Instant;
import java.util.concurrent.atomic.AtomicBoolean;

@Slf4j
@Component
@RequiredArgsConstructor
public class ArchiveCronJob {

```

```

private final ArchiverProperties properties;
private final MongoTemplate mongoTemplate;
private final ArchiveLogger archiveLogger;

private final AtomicBoolean running = new AtomicBoolean(false);

@PostConstruct
public void init() {
    archiveLogger.ensureIndexes();
    log.info("[ArchiveCronJob] Initialised. Schedule='{}' timezone='{}'",
        properties.getSchedule(), properties.getTimezone());
    log.info("[ArchiveCronJob] {} job(s) configured: {}",
        properties.getJobs().size(),

properties.getJobs().stream().map(ArchiverProperties.JobProperties::getName).toList
    }

    @Scheduled(cron = "${archiver.schedule}", zone = "${archiver.timezone}")
    public void runScheduled() {
        execute("scheduled");
    }

    public void runNow() {
        execute("manual");
    }

    private void execute(String trigger) {
        if (!running.compareAndSet(false, true)) {
            log.warn("[ArchiveCronJob] Previous run still in progress – skipping
{} trigger", trigger);
            return;
        }
        long start = System.currentTimeMillis();
        log.info("[ArchiveCronJob] Run started at {} (trigger: {})",
Instant.now(), trigger);
        try {
            for (ArchiverProperties.JobProperties jobConfig :
properties.getJobs()) {
                try {
                    new Archiver(jobConfig, mongoTemplate, archiveLogger).run();
                } catch (Exception e) {
                    log.error("[ArchiveCronJob] Unhandled error in job '{}': {}",
                        jobConfig.getName(), e.getMessage(), e);
                }
            }
        } finally {
            running.set(false);

```



```

        log.info("[ArchiveCronJob] Completed in {}",
            String.format("%.1f", (System.currentTimeMillis() - start) /
1000.0));
    }
}
}

```

```

=====
FILE: src/main/java/com/archiver/model/ArchiveLogEntry.java
=====

```

```

package com.archiver.model;

import lombok.Builder;
import lombok.Data;

import java.util.Date;
import java.util.Map;

/**
 * One archiveLog document per entity per run.
 *
 * <p>Overall status is derived from the per-collection results:
 * <ul>
 *   <li>{@code SUCCESS}           - every collection was SUCCESS or
NO_DOCUMENTS</li>
 *   <li>{@code PARTIAL_FAILURE} - at least one FAILED, at least one
SUCCESS/NO_DOCUMENTS</li>
 *   <li>{@code FAILED}           - every collection failed</li>
 * </ul>
 *
 * <p>Document shape in MongoDB:
 * <pre>
 * {
 *   "jobName"    : "orders",
 *   "entityId"   : "ORD-7823",
 *   "status"     : "PARTIAL_FAILURE",
 *   "collections": {
 *     "orderItems"      : "SUCCESS",
 *     "orderPayments"   : "SUCCESS",
 *     "orderEvents"     : "NO_DOCUMENTS",
 *     "orderNotifications": "FAILED",
 *     "orders"          : "SKIPPED"
 *   },
 *   "runAt"       : ISODate("2024-04-15T02:04:31Z"),
 *   "retryCount"  : 1,
 *   "createdAt"  : ISODate("2024-04-15T02:04:31Z")
 * }
 *

```

```

* }
* </pre>
*/
@Data
@Builder
public class ArchiveLogEntry {

    public enum Status {
        /** Every collection succeeded or had no documents. */
        SUCCESS,
        /** At least one collection failed, at least one succeeded. */
        PARTIAL_FAILURE,
        /** Every collection failed – anchor was never attempted. */
        FAILED,
        /** Anchor was not attempted because a related collection failed. */
        SKIPPED
    }

    public enum CollectionStatus {
        SUCCESS,
        FAILED,
        NO_DOCUMENTS,
        /** Anchor skipped because a related collection failed. */
        SKIPPED
    }

    private String jobName;
    private String entityId;

    /** Rolled-up status derived from all collection outcomes. */
    private Status status;

    /** Per-collection outcome keyed by collection name. */
    private Map<String, String> collections;

    private Date runAt;
}

```

```

=====
FILE: src/main/java/com/archiver/util/ArchiveLogger.java
=====

```

```

package com.archiver.util;

import com.archiver.config.ArchiverProperties;
import com.archiver.model.ArchiveLogEntry;
import lombok.RequiredArgsConstructor;

```

```

import lombok.extern.slf4j.Slf4j;
import org.bson.Document;
import org.springframework.data.domain.Sort;
import org.springframework.data.mongodb.core.MongoTemplate;
import org.springframework.data.mongodb.core.index.CompoundIndexDefinition;
import org.springframework.data.mongodb.core.index.Index;
import org.springframework.data.mongodb.core.query.Criteria;
import org.springframework.data.mongodb.core.query.Query;
import org.springframework.data.mongodb.core.query.Update;
import org.springframework.stereotype.Component;

import java.util.Date;

/**
 * ArchiveLogger
 *
 * <p>Persists one document per entity per run into the {@code archiveLog}
collection.
 * Upsert key is {@code (jobName, entityId)} – re-runs overwrite the previous
entry.
 */
@Slf4j
@Component
@RequiredArgsConstructor
public class ArchiveLogger {

    private final MongoTemplate mongoTemplate;
    private final ArchiverProperties properties;

    /**
     * Upsert one log entry for an entity covering all its collection outcomes.
     * Silently swallows exceptions so logging never crashes the archive job.
     */
    public void log(ArchiveLogEntry entry) {
        try {
            Query query = Query.query(
                Criteria.where("jobName").is(entry.getJobName())
                    .and("entityId").is(entry.getEntityId())
            );

            Update update = new Update()
                .set("status", entry.getStatus().name())
                .set("collections", entry.getCollections())
                .set("runAt", new Date())
                .inc("retryCount", 1)
                .setOnInsert("createdAt", new Date());

```

```

        mongoTemplate.upsert(query, update,
properties.getArchiveLogCollection());

        } catch (Exception e) {
            log.warn("[ArchiveLogger] Failed to write log for entity={}: {}",
                entry.getEntityId(), e.getMessage());
        }
    }

    /**
     * Ensure indexes on the archiveLog collection. Idempotent – safe to call on
     every startup.
     */
    public void ensureIndexes() {
        try {
            String col = properties.getArchiveLogCollection();

            // Unique compound index – upsert key
            mongoTemplate.indexOps(col).ensureIndex(
                new CompoundIndexDefinition(
                    new Document("jobName", 1).append("entityId", 1)
                ).unique()
            );

            // For filtering by overall status + job
            mongoTemplate.indexOps(col).ensureIndex(
                new CompoundIndexDefinition(new Document("status",
1).append("jobName", 1))
            );

            // For time-based auditing
            mongoTemplate.indexOps(col).ensureIndex(
                new Index("runAt", Sort.Direction.DESC)
            );

            log.info("[ArchiveLogger] Indexes ensured on '{}'", col);
        } catch (Exception e) {
            log.warn("[ArchiveLogger] Index creation warning: {}",
e.getMessage());
        }
    }
}

```

```

=====
FILE: src/main/java/com/archiver/util/IdPathResolver.java
=====

```

```

package com.archiver.util;

import org.bson.Document;
import org.springframework.data.mongodb.core.query.Criteria;
import org.springframework.data.mongodb.core.query.Query;

import java.util.Date;
import java.util.List;

public final class IdPathResolver {

    private IdPathResolver() {}

    public static Object resolveValue(Document doc, String path) {
        String[] parts = path.split("\\.");
        Object current = doc;
        for (String part : parts) {
            if (!(current instanceof Document nested)) return null;
            current = nested.get(part);
        }
        return current;
    }

    public static Query buildSingleQuery(String idPath, Object idValue) {
        return Query.query(Criteria.where(idPath).is(idValue));
    }

    public static Query buildBatchQuery(String idPath, List<?> idValues) {
        return Query.query(Criteria.where(idPath).in(idValues));
    }

    public static Query buildCutoffQuery(String tsField, Date cutoff) {
        return Query.query(Criteria.where(tsField).lt(cutoff));
    }

    public static String extractIdAsString(Document doc, String field) {
        Object value = doc.get(field);
        return value != null ? value.toString() : null;
    }
}

=====
FILE: src/main/java/com/archiver/util/IndexManager.java
=====

package com.archiver.util;

```

```

import lombok.RequiredArgsConstructor;
import lombok.extern.slf4j.Slf4j;
import org.springframework.data.domain.Sort;
import org.springframework.data.mongodb.core.MongoTemplate;
import org.springframework.data.mongodb.core.index.Index;
import org.springframework.data.mongodb.core.index.IndexInfo;
import org.springframework.data.mongodb.core.index.IndexOperations;

import java.util.LinkedHashSet;
import java.util.List;
import java.util.Set;

/**
 * IndexManager
 *
 * Manages temporary indexes for the duration of one archive run.
 * - Pre-existing indexes are reused and never dropped.
 * - Newly created indexes are prefixed with _arch_tmp_ and dropped in
dropCreatedIndexes().
 * - dropCreatedIndexes() must be called in a finally block to guarantee cleanup
on failure.
 */
@Slf4j
@RequiredArgsConstructor
public class IndexManager {

    private static final String INDEX_PREFIX = "_arch_tmp_";

    private final MongoTemplate mongoTemplate;
    private final Set<String> ownedIndexes = new LinkedHashSet<>();

    public void ensureIndex(String collection, String fieldPath) {
        try {
            IndexOperations indexOps = mongoTemplate.indexOps(collection);

            if (indexAlreadyExists(indexOps, fieldPath)) {
                log.info("[IndexManager] Pre-existing index on '{}.{}' – reusing,
will not drop",
                    collection, fieldPath);
                return;
            }

            String indexName = INDEX_PREFIX + fieldPath.replace(".", "_");
            indexOps.ensureIndex(
                new Index().on(fieldPath, Sort.Direction.ASC).named(indexName)
            );

```

```

        ownedIndexes.add(collection + "::" + indexName);
        log.info("[IndexManager] Created temporary index '{}' on '{}.{}'.'",
            indexName, collection, fieldPath);

    } catch (Exception e) {
        log.warn("[IndexManager] Could not ensure index on '{}.{}': {}",
            collection, fieldPath, e.getMessage());
    }
}

public void dropCreatedIndexes() {
    if (ownedIndexes.isEmpty()) return;

    log.info("[IndexManager] Dropping {} temporary index(es)...",
ownedIndexes.size());
    for (String key : ownedIndexes) {
        String[] parts = key.split("::", 2);
        String collection = parts[0];
        String indexName = parts[1];
        try {
            mongoTemplate.indexOps(collection).dropIndex(indexName);
            log.info("[IndexManager] Dropped '{}' on '{}'", indexName,
collection);
        } catch (Exception e) {
            log.warn("[IndexManager] Failed to drop '{}' on '{}': {}",
                indexName, collection, e.getMessage());
        }
    }
    ownedIndexes.clear();
}

public int ownedCount() {
    return ownedIndexes.size();
}

private boolean indexAlreadyExists(IndexOperations indexOps, String fieldPath)
{
    try {
        List<IndexInfo> existing = indexOps.getIndexInfo();
        return existing.stream()
            .anyMatch(info -> info.getIndexFields().stream()
                .anyMatch(f -> f.getKey().equals(fieldPath)));
    } catch (Exception e) {
        log.warn("[IndexManager] Could not read index info: {}",
e.getMessage());
        return false;
    }
}

```

```
}  
}
```

```
=====  
END OF FILE  
=====
```